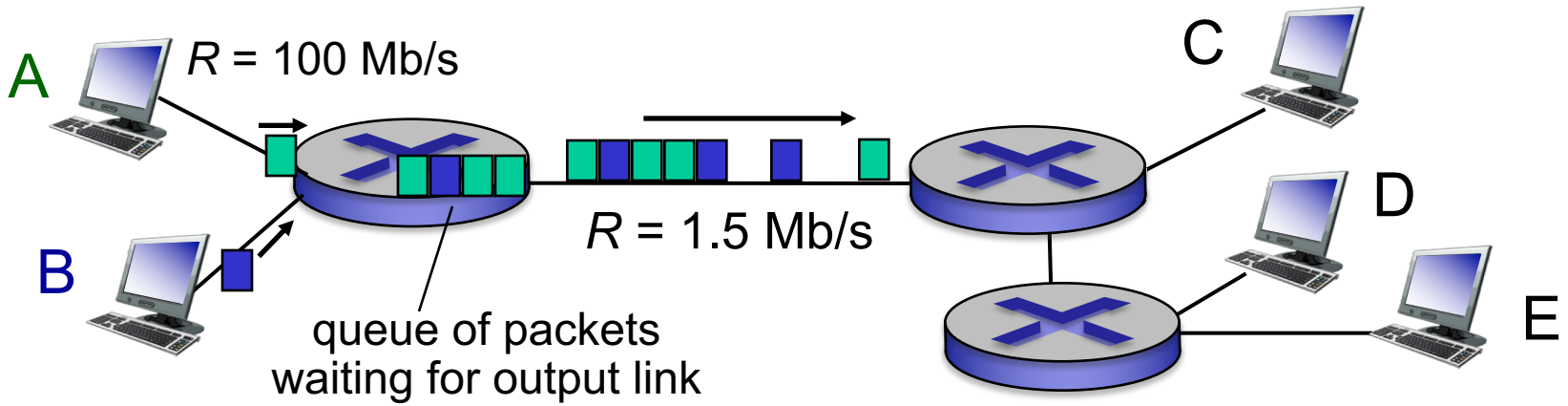
Packet Switching: queueing delay, loss



queuing and loss:

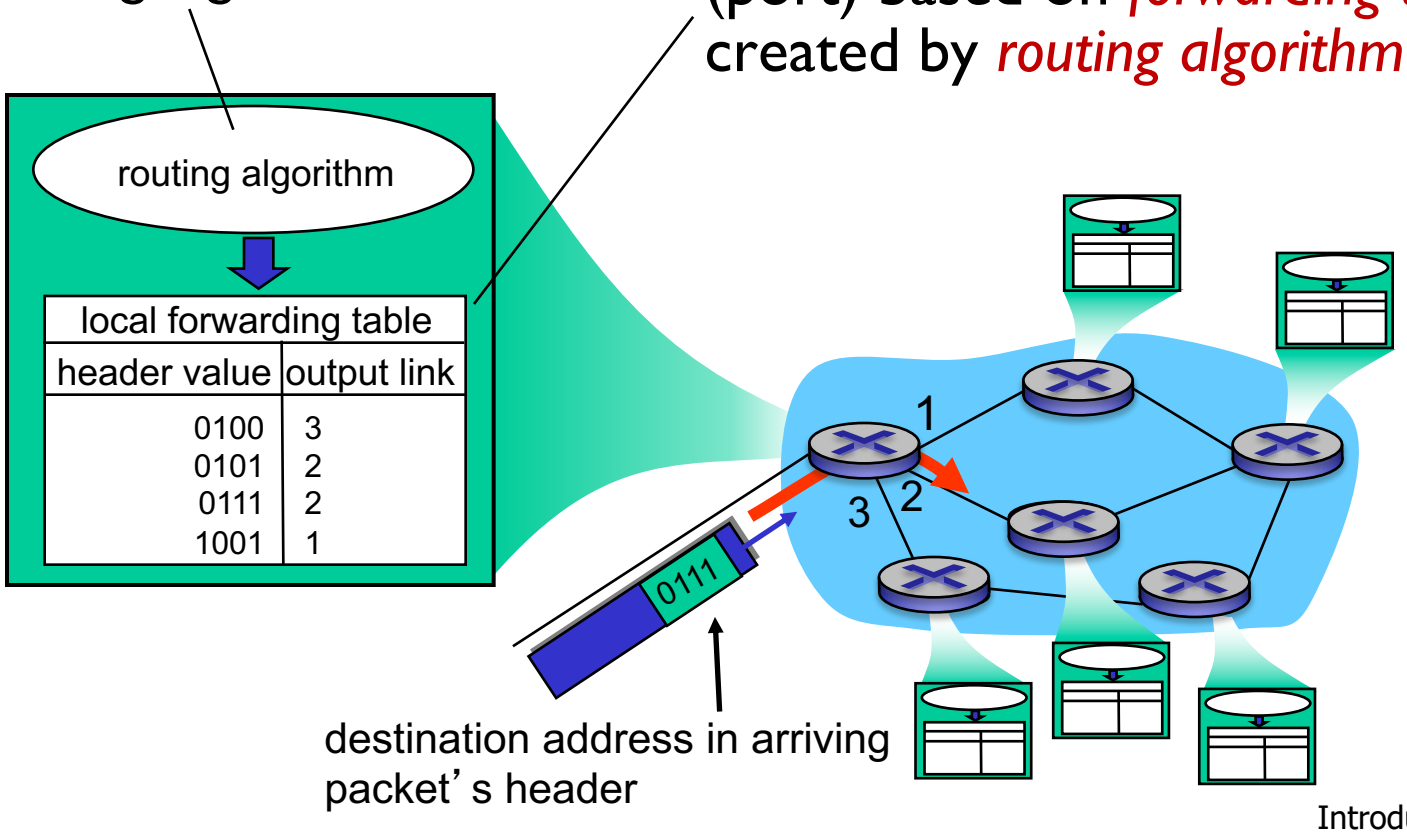
- if arrival rate (in bits) to link exceeds transmission rate of link for a period of time:
  - packets will queue, wait to be transmitted on link
  - packets can be dropped (lost) if memory (buffer) fills up

Two key network-core functions

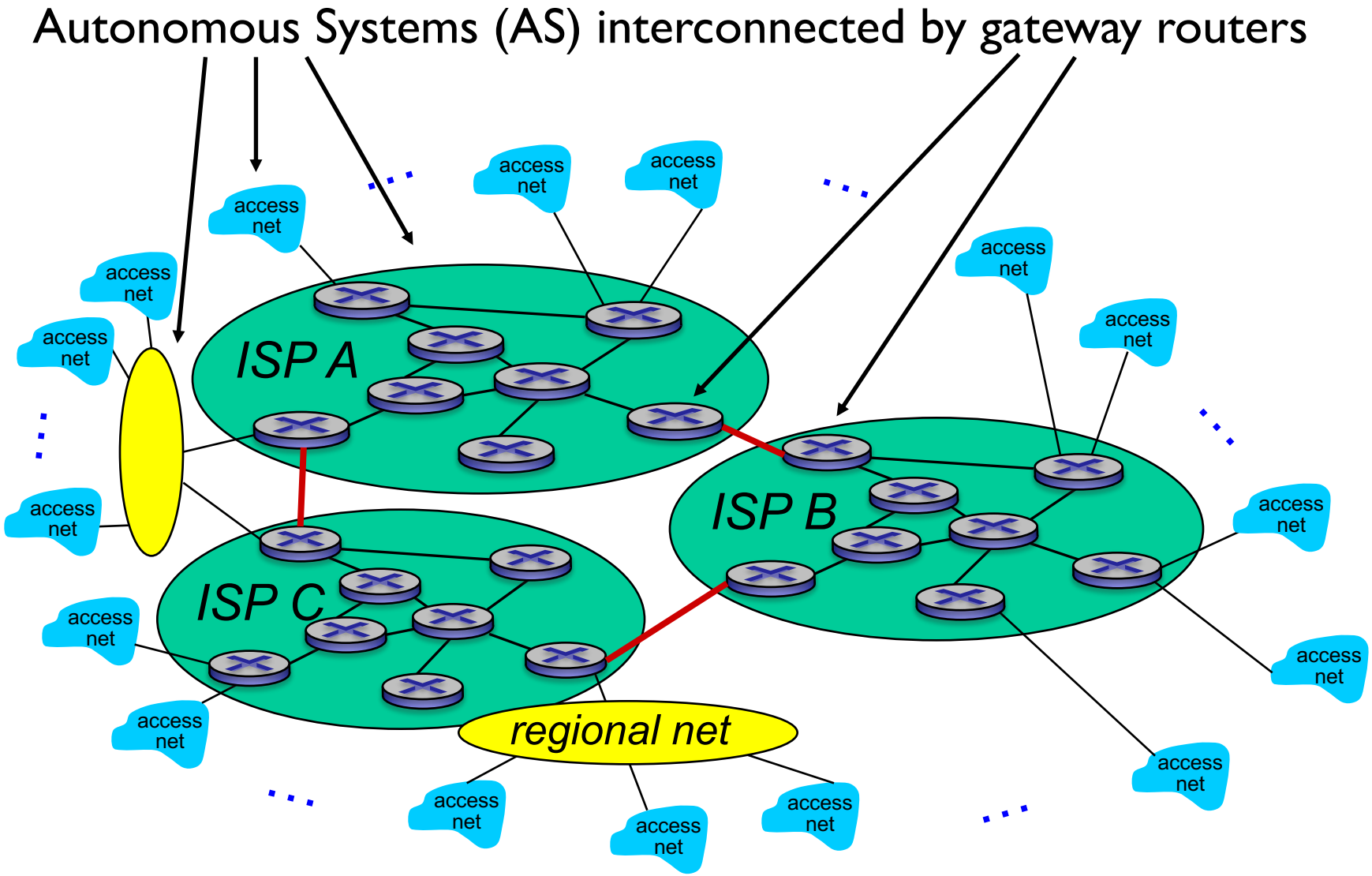
routing: determines source-destination route taken by packets

- *routing algorithms*

forwarding: move packets from router's input (port) to appropriate router output (port) based on *forwarding table* created by *routing algorithm*



Internet structure: network of networks



Chapter 1: roadmap

1.1 what *is* the Internet?

1.2 network edge

- end systems, access networks, links

1.3 network core

- packet switching, network structure

1.5 protocol layers, service models

Protocol "layers"

Networks are complex, with many "pieces":

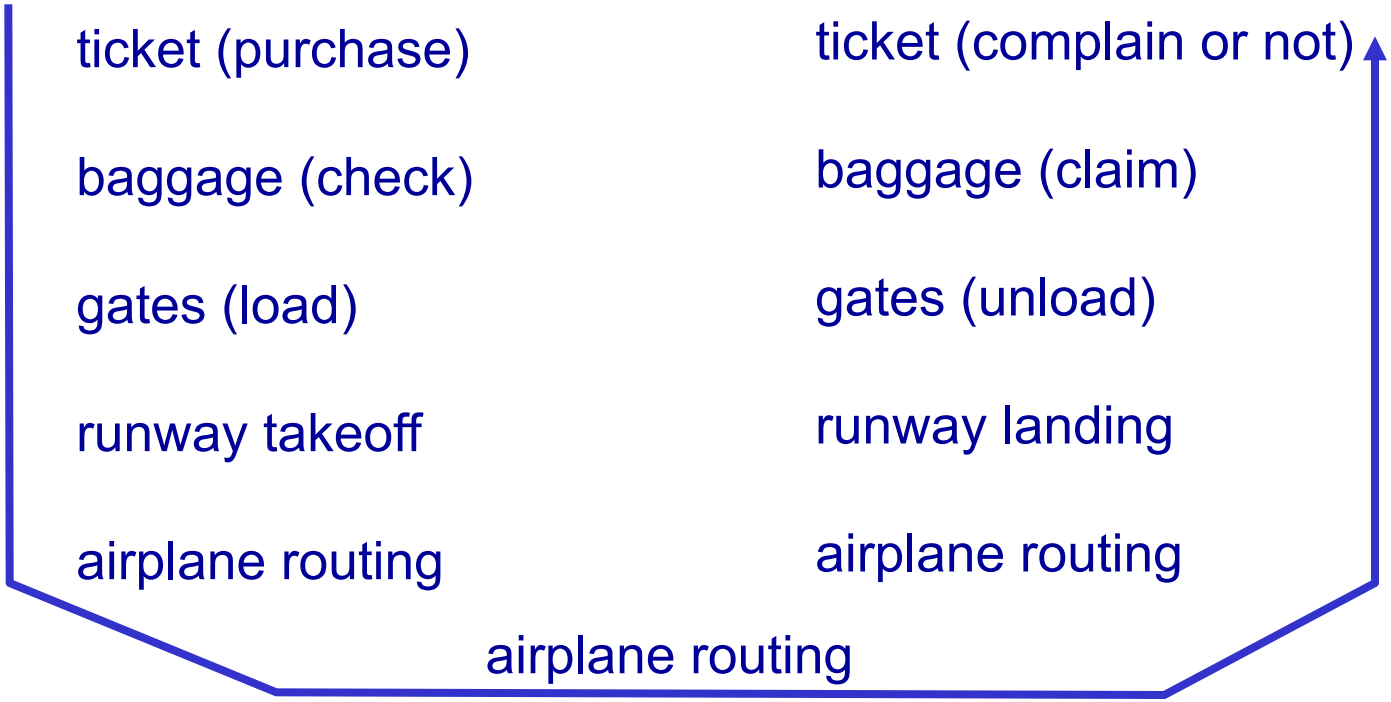
- hosts
- routers
- links of various media
- applications
- protocols
- hardware, software

Question:

how should we organize such a complex system?

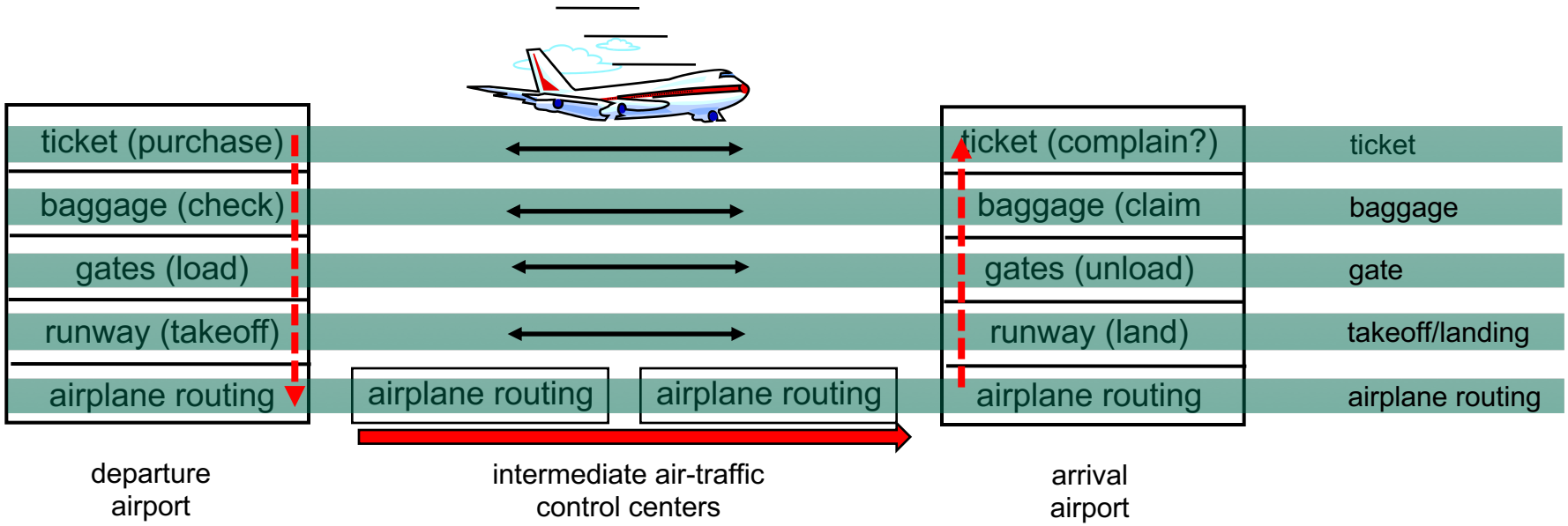
…. or at least our discussion of networks?

Analogy: Organization of air travel



- a series of steps

Layering of airline functionality



layers: each layer implements a service

- via its own internal-layer actions
- relying on services provided by layer below

Why layering?

- *layered* structure allows breakdown of a complex system into smaller, well identified *modules* with explicit responsibilities/relationships to each other
  - layered reference model – *up/down relationships only - well defined*
- layering considered harmful?
  - Overlap in functionality (e.g., error control), dependency wrt specific requirements (e.g., timestamps) – not fully independent

Why modules?

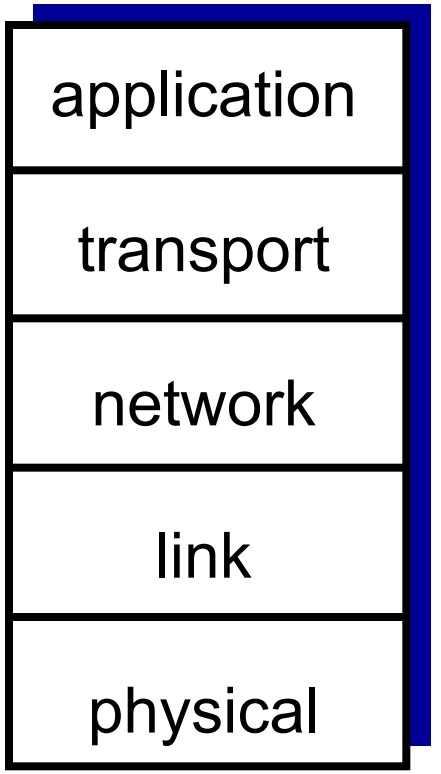
- *modularization* eases maintenance/updating of system
  - change of implementation of a layer's service transparent to rest of system – internals are hidden
    - e.g., change in gate procedure doesn't affect rest of system *operation*, a flight is still a flight, passenger still gets to destination
    - but a change in a layer's service could impact *performance* of end system -> passengers may not be happy with a new gate procedure – unless reflected in their ticket purchase

Is layering the right choice?

- is layering harmful?
  - overlap in functionality (e.g., error control),
  - dependency with respect to specific requirements (e.g., timestamps) – not fully independent

Internet protocol stack - layers

- *application:* supporting networked applications, process to process transfer – data stream → managed as *messages*
  - FTP, SMTP, HTTP,….
- *transport:* process to process message transfer → managed as *segments/datagrams*
  - TCP, UDP
- *network:* routing of data segments from source to destination hop by hop → managed as *datagrams*
  - IP (Internet Protocol)
- *link:* data transfer between neighboring network elements → managed as *frames*
  - Ethernet, 802.11 (WiFi), PPP
- *physical:* data "on the wire" → managed as *bits*
  - optical, electrical, electromagnetic…..



Layers: Functionality & Services

- Data Link Layer: Transmission of frames
  - Functions: Framing, media access control, error checking, flow control
- Network Layer: Forwarding of packets (datagrams)
  - Functions: Network addressing, hop by hop routing, header checking, loop prevention
- Transport Layer: Transfer of "chunks of application data"
  - Functions: Connection establishment/termination, error control, flow control, congestion control
- Application Layer: Application specific – display/presentation
  - Functions: Synchronization/timing, error recovery